Elasticsearch para Aplicações RAG

Do Zero ao Código Python

Ciro Alves Justino Nunes

4 de fevereiro de 2026

Resumo

Este documento apresenta um guia prático e teórico completo sobre a implementação de busca e RAG (Retrieval-Augmented Generation) com Elasticsearch, abordando desde conceitos fundamentais e comandos no Kibana até a implementação final em Python com algoritmos de reranking.

Sumário

1 O Índice e Analisadores **3**
1.1 Limpando o ambiente (Reset) . . . 3
1.2 Testando Analisadores (Antes de Criar) . . . 3
1.3 Criando o Índice (Settings & Mappings) . . . 4

2 CRUD - Criar, Ler, Atualizar e Deletar **6**
2.1 Indexando um Documento (Create) . . . 6
2.2 Atualizações (Update) . . . 6
2.2.1 Atualização Parcial (Recomendado) . . . 6
2.2.2 Substituição Total (Full Update) . . . 6
2.3 Deletando (Delete) . . . 7

3 Busca Textual (Léxica) **8**
3.1 Busca Multi-Campos (A que usamos na App) . . . 8

4 Busca Vetorial (Semântica) **9**
4.1 O Poder dos Vetores . . . 9

5 Depara - Kibana para Python **10**
5.1 Conexão (IndexerService.__init__) . . . 10
5.2 Criação do Índice (setup_index) . . . 10
5.3 Indexação em Lote (ingest_file / _process_text) . . . 10
5.4 A Busca (search) . . . 11
5.4.1 Busca Léxica (app/services/indexer.py:326) . . . 11
5.4.2 Busca Vetorial (app/services/indexer.py:319) . . . 11

6 Deep Dive - Código de Busca e Rerank **12**
6.1 Algoritmo de Busca Completo . . . 12

6.2 Deep Dive: Reciprocal Rank Fusion (RRF) 13
6.3 Deep Dive: Weighted Linear Fusion (WLF) 14

7 Conclusão e Próximos Passos **16**
7.1 Recapitulando 16
7.2 Próximos Passos 16

Introdução

Este documento é um **guia prático e teórico** para entender como a nossa aplicação RAG funciona "por baixo do capô". Ele foi desenhado para você copiar os comandos e colar diretamente no **Console do Kibana (Dev Tools)**.

Para este laboratório, usaremos um índice de teste chamado `documentos_teste` e vetores simplificados (apenas 3 dimensões) para facilitar a visualização. Na aplicação real, usamos 1536 dimensões.

1 O Índice e Analisadores

Antes de guardar dados, precisamos ensinar o Elasticsearch a "ler"Português. Isso é feito através de **Analyzers**.

1.1 Limpando o ambiente (Reset)

Vamos garantir que começamos do zero.

```
DELETE /documentos_teste
```

Listing 1: Limpando o índice

1.2 Testando Analisadores (Antes de Criar)

Veja como o Elasticsearch entende a frase "Relatórios Financeiros 2024". O analisador `standard` (padrão) apenas quebra espaços.

```
POST /_analyze
{
  "tokenizer": "standard",
  "text": "Relat rios Financeiros 2024"
}
```

Listing 2: Teste do analisador standard

Agora, vamos simular o nosso analisador customizado de **Português** (que remove acentos, coloca em minúsculo e remove plural/sufixos).

```
POST /_analyze
{
  "tokenizer": "standard",
  "filter": ["lowercase", "asciifolding", {"type":"stemmer", "language":
   "light_portuguese"}],
  "text": "Relat rios Financeiros 2024"
}
```

Listing 3: Teste do analisador português

> **Resultado Esperado**: Você verá `relatori`, `financeir`. Isso permite que buscando "relatório"você encontre "relatórios".

1.3 Criando o Índice (Settings & Mappings)

Agora vamos criar o índice `documentos_teste`. **Diferença Importante Aqui**: Vamos definir `dims: 3` para o vetor, só para podermos brincar manualmente. Na app real, usamos `1536`.

```
PUT /documentos_teste
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "analysis": {
      "char_filter": {
        "protect_codes": {
          "type": "pattern_replace",
          "pattern": "(\\b[A-Z]{2,}\\d+[A-Z]*\\d*\\b)",
          "replacement": " $1 "
        }
      },
      "filter": {
        "portuguese_stop": {
          "type": "stop",
          "stopwords": "_portuguese_"
        },
        "pt_light_stemmer": {
          "type": "stemmer",
          "language": "light_portuguese"
        },
        "ngram_filter": {
          "type": "edge_ngram",
          "min_gram": 3,
          "max_gram": 20
        }
      },
      "analyzer": {
        "pt_analyzer": {
          "type": "custom",
          "char_filter": ["protect_codes"],
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "portuguese_stop", "
   pt_light_stemmer"]
        },
        "ngram_analyzer": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "ngram_filter"]
        },
        "exact_analyzer": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "portuguese_stop"]
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "content": {
```

```
        "type": "text",
        "analyzer": "pt_analyzer",
        "search_analyzer": "pt_analyzer",
        "fields": {
          "suggest": {
            "type": "text",
            "analyzer": "ngram_analyzer"
          },
          "exact": {
            "type": "text",
            "analyzer": "exact_analyzer"
          }
        }
      },
      "embedding": {
        "type": "dense_vector",
        "dims": 3,
        "index": true,
        "similarity": "cosine"
      },
      "title": {
        "type": "text",
        "analyzer": "pt_analyzer",
        "fields": {
          "exact": {
            "type": "text",
            "analyzer": "exact_analyzer"
          }
        }
      },
      "source": { "type": "keyword" },
      "file_path": { "type": "keyword" },
      "file_type": { "type": "keyword" },
      "chunk_id": { "type": "keyword" },
      "doc_id": { "type": "keyword" },
      "created_at": { "type": "date" }
    }
  }
}
```

Listing 4: Criando o índice com configurações completas

2 CRUD - Criar, Ler, Atualizar e Deletar

2.1 Indexando um Documento (Create)

Vamos inserir dados. Imagine que o vetor `[0.1, 0.5, 0.9]` representa o "sentido"deste texto.

```
POST /documentos_teste/_doc/1
{
  "doc_id": "doc-01",
  "chunk_id": "chunk_0",
  "title": "Balan o 2024",
  "content": "O lucro da empresa subiu drasticamente.",
  "source": "balanco.pdf",
  "file_path": "/uploads/balanco.pdf",
  "created_at": "2025-01-15T10:00:00",
  "embedding": [0.1, 0.5, 0.9]
}
```

Listing 5: Indexando documento 1 (Finanças)

Vamos inserir um segundo documento para comparação.

```
POST /documentos_teste/_doc/2
{
  "doc_id": "doc-02",
  "chunk_id": "chunk_0",
  "title": "Receita de Bolo",
  "content": "Misture farinha e ovos. Asse por 40 minutos.",
  "source": "receita.txt",
  "created_at": "2025-01-16T10:00:00",
  "embedding": [0.9, 0.1, 0.1]
}
```

Listing 6: Indexando documento 2 (Culinária)

2.2 Atualizações (Update)

2.2.1 Atualização Parcial (Recomendado)

Imagine que queremos apenas corrigir o título do `doc-01`, sem reenviar o texto todo ou o vetor. Usamos o endpoint `_update` com `doc`.

```
POST /documentos_teste/_update/1
{
  "doc": {
    "title": "Balan o Financeiro Consolidado 2024"
  }
}
```

Listing 7: Atualização parcial

2.2.2 Substituição Total (Full Update)

Se você fizer um `PUT` ou `POST` no mesmo ID sem usar `_update`, você **substitui** o documento inteiro. Cuidado! Se não mandar o vetor, ele some.

```
PUT /documentos_teste/_doc/1
{
  "doc_id": "doc-01",
  "chunk_id": "chunk_0",
  "title": "Balan o Financeiro Consolidado 2024",
  "content": "O lucro da empresa subiu drasticamente.",
  "source": "balanco.pdf",
  "file_path": "/uploads/balanco.pdf",
  "created_at": "2025-01-15T10:00:00",
  "embedding": [0.1, 0.5, 0.9]
}
```

Listing 8: Substituição total (Perigo a vista)

2.3 Deletando (Delete)

```
DELETE /documentos_teste/_doc/2
```

Listing 9: Deletando um documento

(Não delete agora se quiser testar a busca abaixo!)

3 Busca Textual (Léxica)

3.1 Busca Multi-Campos (A que usamos na App)

Buscando por termos. Note que "financeiro"está no título atualizado, mas não estava no original. O Elastic já sabe.

```
GET /documentos_teste/_search
{
  "query": {
    "bool": {
      "should": [
        {
          "multi_match": {
            "query": "lucro financeiro",
            "fields": ["title^4", "content^2", "source^3"],
            "type": "best_fields"
          }
        }
      ]
    }
  }
}
```

Listing 10: Exemplo de Multi Match Query

4 Busca Vetorial (Semântica)

4.1 O Poder dos Vetores

- Documento 1 (Finanças): `[0.1, 0.5, 0.9]`
- Documento 2 (Bolo): `[0.9, 0.1, 0.1]`

Se eu buscar por "dinheiro"(conceito financeiro), digamos que a IA gerou o vetor `[0.2, 0.4, 0.8]`. Ele está matematicamente perto do Doc 1 e longe do Doc 2.

```
GET /documentos_teste/_search
{
  "knn": {
    "field": "embedding",
    "query_vector": [0.2, 0.4, 0.8],
    "k": 2,
    "num_candidates": 10
  },
  "_source": ["title", "content", "score"]
}
```

Listing 11: Exemplo de kNN Search

Resultado: O Elastic vai retornar o "Balanço"com um score alto, pois os números são próximos.

5 Depara - Kibana para Python

Agora vamos conectar tudo o que vimos com o código real em `app/services/indexer.py`. A biblioteca oficial `elasticsearch` do Python mapeia esses comandos JSON quase 1:1.

5.1 Conexão (IndexerService.__init__)

No Kibana, você apenas abre o console. No Python, precisamos autenticar.

```
# indexer.py (Linhas 112-118)
self.es = Elasticsearch(
    settings.ELASTIC_HOST, # ex: https://localhost:9200
    basic_auth=(user, password),
    # ...
)
```

Listing 12: Conexão Python

5.2 Criação do Índice (setup_index)

O JSON gigante do Capítulo 1 vira um dicionário Python.

```
# indexer.py (Linhas 21-107 define o dicion rio ES_MAPPING)

# O comando PUT /documents vira:
self.es.indices.create(
    index=INDEX_NAME,
    settings=ES_MAPPING["settings"],
    mappings=ES_MAPPING["mappings"]
)
```

Listing 13: Criando índice via Python

5.3 Indexação em Lote (ingest_file / _process_text)

Ao invés de fazer um `POST` por vez, usamos o **Bulk API** para performance.

```
# indexer.py (Loop nas linhas 251-283)

actions = []
for chunk in chunks:
    # Cria o dicion rio que representa o JSON do Cap tulo 2.1
    doc = {
        "_index": INDEX_NAME,
        "_source": {
            "content": chunk_text,
            "embedding": vetor_openai, # vetor real de 1536 dimens es
            "title": ...,
            # ...
        }
    }
    actions.append(doc)

# Envia tudo de uma vez
helpers.bulk(self.es, actions)
```

Listing 14: Bulk Indexing Python

5.4 A Busca (search)

Esta é a parte mais crítica. O método `search` constrói o JSON do Capítulo 3 e 4 dinamicamente.

5.4.1 Busca Léxica (app/services/indexer.py:326)

```
keyword_query = {
    "bool": {
        "should": [
             # Equivalente ao multi_match do Cap tulo 3.1
            {"multi_match": {"query": query, "fields": ["content.exact
   ^10", ...], ...}},
            {"wildcard": ...} # Equivalente ao wildcard do Cap tulo 3.1
        ]
    }
}
# Executa:
self.es.search(index=INDEX_NAME, query=keyword_query, ...)
```

Listing 15: Busca Léxica Python

5.4.2 Busca Vetorial (app/services/indexer.py:319)

```
knn_query = {
    # Equivalente ao JSON do Cap tulo 4.1
    "field": "embedding",
    "query_vector": query_vector, # Vetor que veio da OpenAI
    "k": 20,
    "num_candidates": 100
}
# Executa:
self.es.search(index=INDEX_NAME, knn=knn_query, ...)
```

Listing 16: Busca Vetorial Python

6 Deep Dive - Código de Busca e Rerank

Para que não reste nenhuma dúvida, apresentamos abaixo o código consolidado e comentado da lógica de busca e reranking (reordenação) da aplicação.

6.1 Algoritmo de Busca Completo

Este método decide se usa OpenAI ou não, executa as buscas (léxica e/ou vetorial) e chama o reranking.

```
def search(self, query: str, method: str = "hybrid", rerank_method: str = "RRF",
    doc_id_filter: str = None):
    """
    Orquestrador de Busca: H brida , L xica ou Sem ntica.
    """
    openai_available = self.is_openai_available

    # Se OpenAI cair, for amos modo l xico (fallback de seguran a)
    if not openai_available and method != "lexical":
        method = "lexical"

    # Gera embedding apenas se necess rio (custo/performance)
    query_vector = self.get_embedding(query) if method in ["semantic", "hybrid"] else
    None

    # Defini  o dos campos que queremos retornar do Elastic (SELECT fields)
    _source_fields = ["content", "title", "score", "chunk_id", "doc_id", "source", "
    file_path", "file_type", "created_at"]

    # 1. Configurar Query Vetorial (kNN)
    knn_query = {
        "field": "embedding",
        "query_vector": query_vector,
        "k": 20, # Top 20 vizinhos mais pr ximos
        "num_candidates": 100 # Explora 100 para achar os 20 melhores
    }

    # 2. Configurar Query L xica (Keyword)
    keyword_query = {
        "bool": {
            "should": [
                # Busca exata (boost alto ^20)
                {"multi_match": {"query": query, "fields": ["content.exact^10", "title.
    exact^20", "source^15"], "type": "best_fields"}},
                # Busca aproximada (boost m dio)
                {"multi_match": {"query": query, "fields": ["content^2", "title^4", "
    source^3", "file_path^3"], "type": "best_fields"}},
                # Busca "fuzzy" (toler ncia a erro de digita  o)
                {"multi_match": {"query": query, "fields": ["title^2", "source^2"], "
    fuzziness": "AUTO", "prefix_length": 2}},
                # Busca coringa (partes da palavra)
                {"wildcard": {"source": f"*{query}*"}},
                {"wildcard": {"file_path": f"*{query}*"}}
            ]
        }
    }

    # 3. Execu  o das Buscas no Elasticsearch
    lexical_results = None
    semantic_results = None

    if method in ["lexical", "hybrid"]:
        lexical_results = self.es.search(
            index=INDEX_NAME,
            query=keyword_query,
            source=_source_fields,
            size=20     # Trazemos 20 candidatos l xicos
        )
```

```
    if method in ["semantic", "hybrid"]:
        semantic_results = self.es.search(
            index=INDEX_NAME,
            knn=knn_query,
            source=_source_fields,
            size=20    # Trazemos 20 candidatos sem nticos
        )

    # 4. Combina  o dos Resultados (Rerank)
    if method == "hybrid":
        if rerank_method == "RRF":
            results = self._manual_rrf(semantic_results, lexical_results, k=60)
        else: # WLF
            results = self._manual_wlf(semantic_results, lexical_results, alpha=0.5)
    elif method == "semantic":
        results = semantic_results
    else: # lexical
        results = lexical_results

    # Formata para o frontend (JSON limpo)
    return {
        "results": self._format_results(results, deduplicate=(method != "lexical")),
        "openai_status": "available" if openai_available else "unavailable"
    }
```

Listing 17: Método search completo

6.2 Deep Dive: Reciprocal Rank Fusion (RRF)

O RRF é um algoritmo "democrático". Ele não se importa com o *valor* do score (que pode ser 10.5 no léxico e 0.92 no vetorial), mas sim com a **posição** no ranking.

Fórmula:

$$Score(d) = \sum_{i \in rankings} \frac{1}{k + rank_i(d)} \tag{1}$$

- $\mathbf{rank_i(d)}$: A posição do documento d na lista i (1º, 2º, 3º...).
- **k**: Uma constante (geralmente 60) que suaviza o impacto dos primeiros lugares.

Exemplo:

- Doc A: 1º no Léxico, não aparece no Vetorial. $\rightarrow$ Score $= 1/(60+1) =$ **0.0163**
- Doc B: 10º no Léxico, 2º no Vetorial. $\rightarrow$ Score $= 1/(60+10) + 1/(60+2) = 0.0142 + 0.0161 =$ **0.0303**

Conclusão: Doc B ganha, pois foi consistente em ambas as listas!

```
def _manual_rrf(self, semantic_results, lexical_results, k=60):
    from collections import defaultdict

    rrf_scores = defaultdict(float)
    doc_data = {}  # Cache dos dados do documento para retornar depois

    # 1. Processa Ranking Sem ntico
    # enumerate(..., start=1) nos d  o Rank (1  , 2  , 3  ...)
    for rank, hit in enumerate(semantic_results.get("hits", {}).get("hits", []), start=1)
    :
        doc_id = hit["_id"]
        rrf_scores[doc_id] += 1.0 / (k + rank)  # Aplica a f rmula
        if doc_id not in doc_data:
            doc_data[doc_id] = hit

    # 2. Processa Ranking L xico
    for rank, hit in enumerate(lexical_results.get("hits", {}).get("hits", []), start=1):
```

```
        doc_id = hit["_id"]
        rrf_scores[doc_id] += 1.0 / (k + rank)  # Soma ao score existente
        if doc_id not in doc_data:
            doc_data[doc_id] = hit
        # Se a busca l xica trouxe highlights (trechos grifados), preservamos eles
        if "highlight" in hit and doc_id in doc_data:
            doc_data[doc_id]["highlight"] = hit["highlight"]

    # 3. Ordena pelo Score Final RRF (Maior    melhor)
    sorted_docs = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)

    # 4. Reconstr i a lista de hits no formato que o frontend espera
    combined_hits = []
    for doc_id, score in sorted_docs[:10]:  # Retorna apenas o Top 10 consolidado
        hit = doc_data[doc_id].copy()
        hit["_score"] = score
        combined_hits.append(hit)

    return {"hits": {"hits": combined_hits, "total": {"value": len(combined_hits)}}}
```

Listing 18: Implementação RRF

6.3 Deep Dive: Weighted Linear Fusion (WLF)

O WLF é uma média ponderada. Aqui nós confiamos no *valor* do score, mas precisamos **normalizá-lo** (colocar entre 0 e 1) porque o Elastic pode retornar scores léxicos infinitos.

Fórmula:

$$Score(d) = \alpha \cdot Norm(Score_{sem}) + (1 - \alpha) \cdot Norm(Score_{lex}) \tag{2}$$

- α **(alpha)**: Peso para a busca semântica (0.0 a 1.0). Se 0.5, é meio a meio.
- **Norm(score)**: Score do documento dividido pelo maior score daquela lista.

```
def _manual_wlf(self, semantic_results, lexical_results, alpha=0.5):
    from collections import defaultdict

    combined_scores = defaultdict(float)
    doc_data = {}

    # 1. Normalizar e Somar Scores Sem nticos
    hits = semantic_results.get("hits", {}).get("hits", [])
    if hits:
        # O melhor score da lista (o primeiro) serve de base para normaliza  o (0.0 a
    1.0)
        max_score = max(hit["_score"] for hit in hits) or 1.0

        for hit in hits:
            doc_id = hit["_id"]
            norm_score = hit["_score"] / max_score  # Normaliza  o
            combined_scores[doc_id] += alpha * norm_score  # Peso Alpha
            doc_data[doc_id] = hit

    # 2. Normalizar e Somar Scores L xicos
    hits = lexical_results.get("hits", {}).get("hits", [])
    if hits:
        max_score = max(hit["_score"] for hit in hits) or 1.0

        for hit in hits:
            doc_id = hit["_id"]
            norm_score = hit["_score"] / max_score
            combined_scores[doc_id] += (1 - alpha) * norm_score  # Peso (1 - Alpha)
            if doc_id not in doc_data:
                doc_data[doc_id] = hit
            if "highlight" in hit and doc_id in doc_data:
                doc_data[doc_id]["highlight"] = hit["highlight"]
```

```python
    # 3. Ordenar e Retornar
    sorted_docs = sorted(combined_scores.items(), key=lambda x: x[1], reverse=True)

    combined_hits = []
    for doc_id, score in sorted_docs[:10]:
        hit = doc_data[doc_id].copy()
        hit["_score"] = score
        combined_hits.append(hit)

    return {"hits": {"hits": combined_hits, "total": {"value": len(combined_hits)}}}
```

Listing 19: Implementação WLF

7 Conclusão e Próximos Passos

Parabéns! Você saiu do zero, rodou comandos "na mão"no Kibana e entendeu como uma aplicação RAG real processa informações.

7.1 Recapitulando

1. **Fundamentos**: Aprendemos que o Elastic não é só um banco, é uma engine de busca que precisa ser configurada (shards, settings).
2. **Análise de Texto**: Viu como o `pt_analyzer` transforma texto bruto em tokens pesquisáveis.
3. **Híbrido é Vida**: Descobriu a força de combinar busca textual (precisa) com vetorial (inteligente).
4. **Matemática do Rerank**: Desmistificou os algoritmos RRF e WLF que garantem que o melhor resultado chegue ao topo.

7.2 Próximos Passos

Se você quiser evoluir ainda mais este projeto, aqui estão algumas sugestões:

- **Ajuste Fino de Pesos**: No WLF, tente mudar o `alpha`. Dê 80% de peso para o vetor (`0.8`) e veja como a busca muda.
- **Novos Tokens**: Tente adicionar sinônimos no analisador (ex: "casa"= "residência").
- **Melhoria de Embeddings**: Que tal testar modelos diferentes da OpenAI ou até modelos Open Source rodando localmente?

O céu é o limite. Agora você tem a base sólida para construir o que quiser!